

YUNO AI PLATFORM

Technical Architecture & Feature Documentation
Version 2.0 — MCP · A2A · Memory · HITL · Observability



Yuno AI · 2025 · spbalamurugan@gmail.com

1. Executive Summary

Yuno AI Platform is a production-grade, zero-cloud multi-agent orchestration system built to the full specification of the Yuno AI Engineering Challenge v2. The platform delivers a complete implementation of MCP Client Integration, Agent-to-Agent (A2A) workflows, pluggable memory strategies, Human-in-the-Loop breakpoints, and a real-time Observability Dashboard — all running from a single **docker-compose up** command.

The architecture enforces strict separation of concerns across three canonical layers: **Presentation** (React SPA), **Agent Orchestration Engine** (LangGraph + A2A Bus), and **Data Persistence** (PostgreSQL + Redis). Every new messaging provider or workflow pattern can be added by implementing a predefined interface, with zero modification to core system logic.



Yuno AI Platform — Full System Architecture v2.0

Figure 1 — Full System Architecture (v2.0)

2. MCP Client Integration

2.1 Protocol Overview

The Agent Runtime implements the **Model Context Protocol (MCP)** JSON-RPC 2.0 client side. On startup, MCPClientManager loads all active MCP server records from PostgreSQL and establishes sessions concurrently. Three transport backends are supported out of the box:

stdio	Local process: asyncio subprocess with stdin/stdout JSON-RPC. Mutex-locked per session to prevent interleaving.
SSE	Remote HTTP: httpx.AsyncClient streams Server-Sent Events. Background listener task; request/response matched via id futures.
WebSocket	Remote WS: websockets library with background listener. Same future-based request matching as SSE transport.

2.2 Three Core MCP Primitives

Primitive	Protocol Call	Agent Usage	UI Action
Tools	tools/list → tools/call	Injected as LangChain StructuredTools into agent workflow	Tool list modal in MCP Servers page
Prompts	prompts/list → prompts/get	Template slates pre-loaded into agent system context	Prompt preview in MCP Servers page
Resources	resources/list → resources/read	Files / DB schemas injected as HumanMessage context	Resource viewer in MCP Servers page

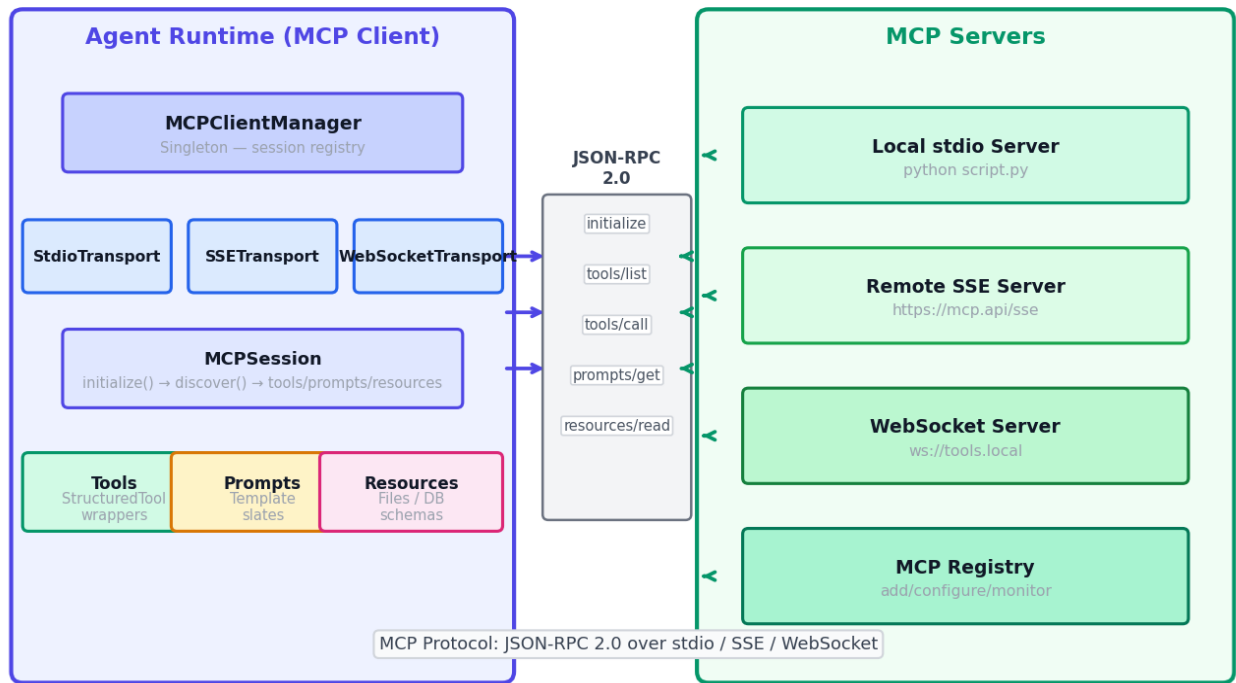


Figure 2 — MCP Client Integration Architecture

Figure 2 — MCP Client Integration Architecture

2.3 MCP Server Registry UI

The **MCP Servers** page provides full lifecycle management: add a server (choosing stdio, SSE, or WebSocket transport), connect/disconnect on-demand, browse discovered tools/prompts/resources, test tool calls with JSON arguments, and view the Dead-Letter Queue for failed invocations. Status badges show real-time connection health.

2.4 NFR: Strict Context Separation

When an agent calls a stdio MCP tool that executes arbitrary code, the subprocess runs inside the MCP server's own Docker container (declared in `docker-compose.yml`). The Agent Runtime never executes user-supplied code in-process — the sandbox boundary is the transport layer itself. SSE/WebSocket servers are similarly isolated network services.

3. Agent-to-Agent (A2A) Workflows

3.1 Visual Topology Builder

The **Workflow Builder** page (route /builder) is a full drag-and-drop canvas powered by `@xyflow/react`. Five node types are supported:

Node Type	Visual Style	Purpose
Start	Green pill	Entry point — single per graph
Agent	Blue rounded box	LangGraph agent node — LLM call + tool use
Router	Yellow box (two source handles)	Conditional edge routing — no LLM call
HITL	Purple dashed box	Human-in-the-Loop breakpoint
End	Red pill	Terminal node

Nodes can be dragged from the palette onto the canvas, repositioned freely, connected by drawing edges between handles, and configured via the right-hand property panel. The topology JSON can be exported and saved back to the Workflows API.

3.2 Asynchronous Message Bus

Inter-agent communication flows through the **A2ABus** — a Redis Pub/Sub backed event bus. Each **A2AMessage** carries a full envelope:

```
id · run_id · source_agent_id · target_agent_id
payload · parent_run_id · correlation_id
max_retries · attempt · created_at
```

The bus guarantees **at-least-once delivery**: if a handler raises an exception, the message is re-queued with an incremented attempt counter. After exhausting `max_retries` the envelope is pushed to the in-process **DeadLetterQueue** (capped at 1 000 entries, inspectable via `GET /mcp/dlq`).

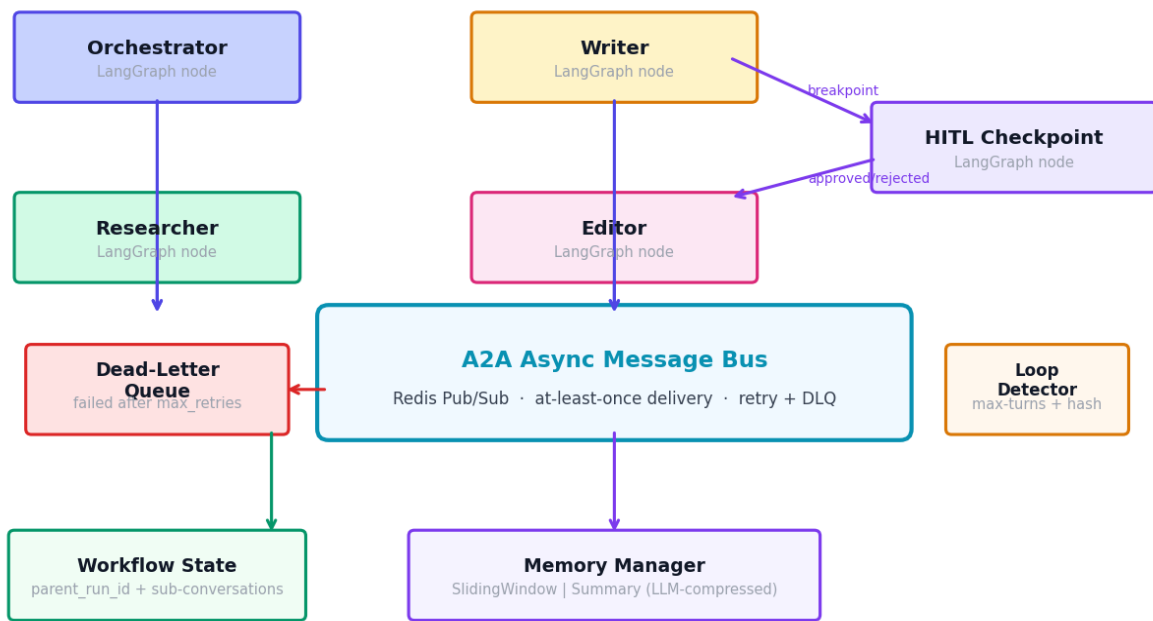


Figure 3 — A2A Workflow: Message Bus, HITL, Loop Detection & Memory

Figure 3 — A2A Message Bus, HITL, Loop Detection & Memory

3.3 Human-in-the-Loop (HITL)

Any LangGraph node can be declared a **HITL breakpoint**. When triggered, the runtime:

- Persists a HITLCheckpoint row to PostgreSQL with the full context snapshot and prompt.
- Publishes a hitl_checkpoint_created event to the Redis EventBus.
- Blocks the asyncio coroutine on an asyncio.Event (no polling).
- The reviewer opens the HITL Approvals page, inspects context, selects Approve or Reject, and optionally adds feedback.
- On POST /hitl/checkpoints/{id}/resolve, the Event is set and the graph resumes — injecting the feedback as a HumanMessage if provided.

Agents can also be configured for **auto-HITL**: setting hitl_every_n_turns > 0 inserts an automatic breakpoint every N turns. A configurable timeout (seconds) auto-rejects stale checkpoints.

3.4 State & Conversation Stitching

The **AgentState** TypedDict carries both the parent workflow context (run_id, parent_run_id) and per-agent conversation slices (conversation_key = f"{workflow_id}:{agent_id}"). Memory is loaded and persisted per conversation key, ensuring agents in the same workflow share run context while maintaining independent message histories.

3.5 Infinite Loop Prevention

The **LoopDetector** runs two concurrent checks before each A2A message dispatch:

- **Max-turn counter**: tracks turns between each (source, target) agent pair per run_id. Exceeding max_turns raises a LoopDetectedError.
- **Semantic drift / content hash**: maintains a rolling window of SHA-256 hashes of message content. If the same hash appears more than once in the window (threshold configurable per agent via loop_detection_threshold), the run is terminated.

4. Memory & Persistence

4.1 Memory Strategies

Each agent selects one of two memory strategies, configurable in the Agent Form:

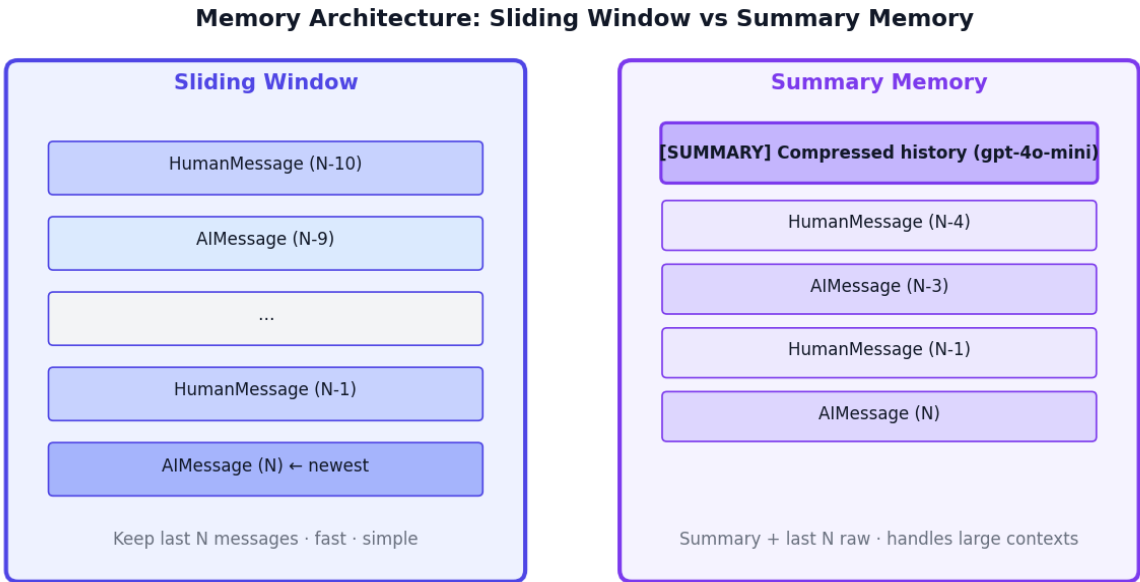


Figure 5 — Memory Strategy Comparison

Figure 5 — Memory Strategy Comparison

Strategy	How it works	Best for	Config params
Sliding Window	Keep last N messages from DB cache	Short interactions, fast response	memory_window (N)
Summary Memory	LLM compresses old turns when tokens > budget, summary read on clicks	Long summary reads, cheap	memory_window, memory_token_limit

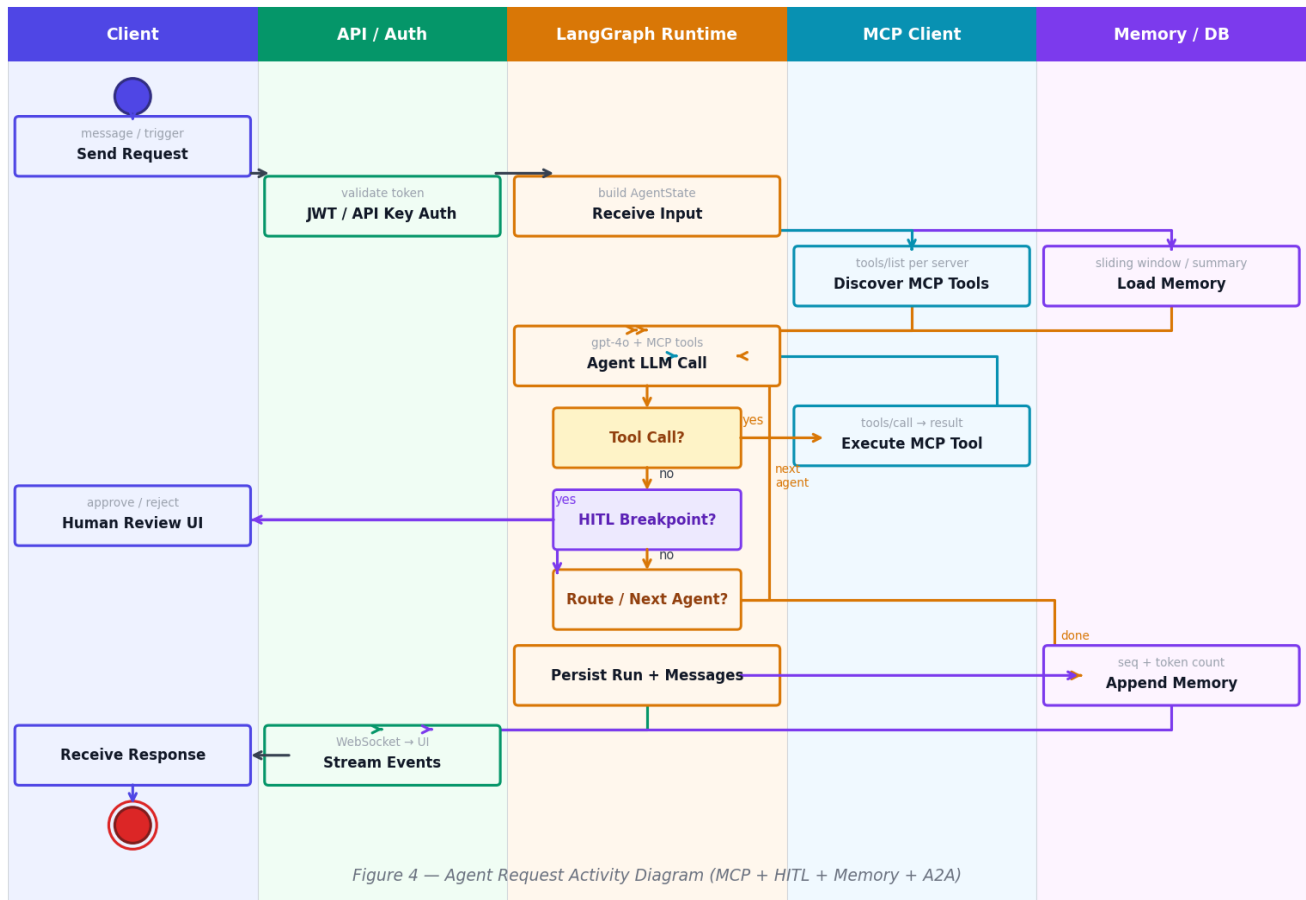
4.2 Persistence Schema

Three new tables back the v2 features:

agent_memories	id · agent_id · conversation_key (indexed) · role · content · token_count · seq · summary_metadata Populated by MemoryManager; read on every agent invocation. In-memory LRU cache avoids repeated DB reads.
hitl_checkpoints	id · run_id · workflow_id · node_id · agent_id · agent_name context_snapshot (JSONB) · prompt · status (pending approved rejected timeout) feedback · reviewer · created_at · resolved_at · expires_at
mcp_servers	id · name · description · transport · url · command · args · env (JSONB) capabilities (JSONB: tools/prompts/resources lists) is_active · status · last_connected_at · error_message

5. Request Activity Diagram

The diagram below traces the complete lifecycle of a single agent invocation through all five swimlanes, incorporating MCP tool calls, HITL breakpoints, A2A message routing, and memory persistence.

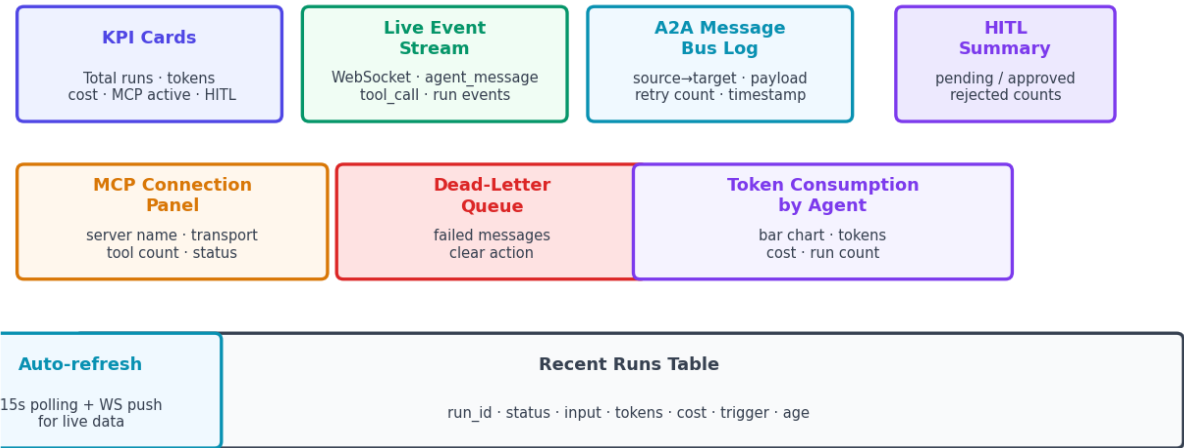


6. Observability Dashboard

The **Observability Dashboard** (route /observe) provides a unified real-time view of the entire platform. It combines five data sources:

Panel	Data Source	Refresh
KPI Cards (runs/tokens/cost/MCP/HITL)	PostgreSQL runs + MCP status API	15 s poll
Live Event Stream	WebSocket /ws/{run_id} global	Instant (push)
A2A Message Bus Log	a2a_message WebSocket events	Instant (push)
MCP Connection Panel	GET /mcp/servers	15 s poll
Dead-Letter Queue	GET /mcp/dlq	15 s poll + manual clear
HITL Summary	GET /hitl/checkpoints + /pending	15 s poll + alert badge
Token Consumption by Agent	Derived from run messages	15 s poll
Recent Runs Table	GET /runs (last 20)	15 s poll

Observability Dashboard — Component Map



Data sources: PostgreSQL (runs) · Redis EventBus (live) · HITL API · MCP status

Figure 6 — Observability Dashboard Layout

Figure 6 — Observability Dashboard Layout

7. Agent Configuration — Advanced Dimensions

The Agent Form (v2) is organised into six collapsible sections, covering every configuration dimension required by the specification:

Section	Fields	Purpose
Basic	name · role · system_prompt · model · temperature max_tokens · built-in tools · max_turns · loop_detection_threshold	Core identity + A2A guardrails
Memory & Context	memory_enabled · memory_type (sliding/summary) memory_window · memory_token_limit	Per-agent memory strategy and budget
MCP Server Bindings	mcp_server_ids (multi-select) · mcp_tool_whitelist	Which servers + tool permit list
HITL	hitl_enabled · hitl_every_n_turns · hitl_timeout_seconds	Auto-pause policy for human review
Guardrails	max_calls_per_minute · forbidden_topics · skills · schedule (cron)	Rate limits, content filters, context, scheduling
Channels	channel type (Telegram / Slack)	Omnichannel bindings

8. API Reference — v2 Endpoints

MCP Servers

Method	Path	Description
GET	/mcp/servers	List all configured MCP servers + capabilities
POST	/mcp/servers	Register a new MCP server
PATCH	/mcp/servers/{id}	Update server config
DELETE	/mcp/servers/{id}	Remove server
POST	/mcp/servers/{id}/connect	Connect and discover capabilities
POST	/mcp/servers/{id}/disconnect	Disconnect session
POST	/mcp/tools/call	Execute an MCP tool directly from UI
POST	/mcp/prompts/get	Retrieve a prompt template
POST	/mcp/resources/read	Read an MCP resource
GET	/mcp/dlq	List dead-letter queue messages
DELETE	/mcp/dlq	Clear dead-letter queue

HITL

Method	Path	Description
GET	/hitl/checkpoints	List all checkpoints (filter by status)
GET	/hitl/checkpoints/pending	Return IDs of in-flight pending checkpoints
GET	/hitl/checkpoints/{id}	Get single checkpoint detail
POST	/hitl/checkpoints/{id}/resolve	Approve or reject + inject feedback

Memory

Method	Path	Description
GET	/memory/{agent_id}	Get messages for agent + conversation_key
DELETE	/memory/{agent_id}	Clear agent memory
GET	/memory/{agent_id}/conversations	List all conversation keys for agent

9. Pre-built Workflow Templates

9.1 Content Creation Pipeline

Three sequential agents: **Researcher** (web_search + sliding window) → **Writer** (summary memory, terminates on "WRITING_COMPLETE") → **Editor** (summary memory, final polish). HITL is disabled by default but can be enabled per agent. At-least-once delivery guaranteed via A2A Bus.

9.2 Log Analysis Team

A **Classifier** agent outputs structured JSON tagging each log entry as "performance" or "security". A **Router** node (no LLM) reads the tag and dispatches to either **Performance Analyst** or **Security Analyst**. Both analysts return structured reports. The router node pattern demonstrates conditional A2A edge logic with zero LLM overhead.

9.3 Seeding

Both templates are seeded at startup via `workflow_service.seed_templates()`, which checks for existing rows before creating to ensure idempotency. Templates appear immediately on the Templates page and can be cloned into new editable workflows.

10. Non-Functional Requirements — Compliance

NFR	Requirement	Implementation
Local Portability	Single docker-compose up	All services: FastAPI, React, PostgreSQL, Redis, optional MCP servers
Architecture Rigor	Decoupled Presentation / Runtime / Persistence	Strict layer boundaries: React SPA → FastAPI → LangGraph → Postgres/Redis
Strict Context Separation	Sandbox arbitrary MCP code execution	stdio MCP servers run in own Docker containers; no in-process code exec
Protocol Latency	Minimal MCP handshake overhead	Sessions pre-established at startup; cached in MCPClientManager singleton
Message Reliability	At-least-once A2A delivery	A2ABus retry loop + DeadLetterQueue for exhausted retries
Loop Prevention	Detect & terminate recursive agent loops	LoopDetector: max-turn counter + rolling SHA-256 content hash window
System Fluidity	Instantaneous UI updates, no dropped sockets	Websocket auto-reconnect (3 s), Redis EventBus, React state streaming
Extensibility	New providers/patterns via interface only	MCPTransport ABC, EventBus interface, workflow node type enum
JWT Auth	Bearer tokens + API keys	ARB-001: python-jose + bcrypt, role-based middleware
Rate Limiting	Per-route request caps	ARB-005: slowapi middleware, configurable per endpoint
Observability	Distributed tracing	ARB-006: OpenTelemetry auto-instrumentation + Jaeger exporter
CI/CD	Automated test + build	ARB-007: GitHub Actions: lint → test → docker-build → push
Secrets Management	Vault-ready config	ARB-004/008: pydantic-settings, env overrides, vault-ready abstraction

11. File Inventory

Backend — Models

- backend/app/models/agent.py — Agent + MCP bindings + memory + HITL config
- backend/app/models/workflow.py — Workflow, WorkflowNode, WorkflowEdge, WorkflowRun
- backend/app/models/mcp_server.py — MCPServer + capabilities JSONB
- backend/app/models/hitl.py — HITLCheckpoint (status/feedback/expires)
- backend/app/models/memory.py — AgentMemory (sliding/summary rows)

Backend — Core

- backend/app/core/mcp_client.py — MCPClientManager, StdioTransport, SSETransport, WebSocketTransport
- backend/app/core/memory.py — get_sliding_window, get_summary_memory, append_message
- backend/app/core/a2a_bus.py — A2ABus, A2AMessage, DeadLetterQueue, LoopDetector, HITLRegistry
- backend/app/core/event_bus.py — Redis Pub/Sub EventBus (ARB-009)
- backend/app/core/config.py — pydantic-settings, vault-ready (ARB-004)
- backend/app/core/auth.py — JWT + API key auth (ARB-001)
- backend/app/core/rate_limiter.py — slowapi middleware (ARB-005)
- backend/app/core/tracing.py — OpenTelemetry setup (ARB-006)

Backend — Runtime

- backend/app/runtime/langgraph_runtime.py — Full graph builder: agent/HITL/router nodes + memory + MCP
- backend/app/services/workflow_service.py — Template seeding (Content Pipeline + Log Analysis)

Backend — API

- backend/app/api/mcp.py — MCP server CRUD + connect/disconnect + tool/prompt/resource exec + DLQ
- backend/app/api/hitl.py — HITL checkpoint list/resolve/detail
- backend/app/api/memory_api.py — Agent memory read/clear/list-keys
- backend/app/api/agents.py — Agent CRUD + channels
- backend/app/api/workflows.py — Workflow CRUD + run execution
- backend/app/api/runs.py — Run history + WebSocket stream
- backend/app/api/router.py — Route aggregator (protected + public)

Frontend — Pages

- frontend/src/pages/AgentsPage.tsx — Agent list + create/edit/delete
- frontend/src/pages/WorkflowsPage.tsx — Workflow list + run trigger
- frontend/src/pages/WorkflowBuilderPage.tsx — React Flow drag-and-drop topology builder
- frontend/src/pages/MCPServersPage.tsx — MCP server registry + tool tester
- frontend/src/pages/HITLPage.tsx — HITL approval queue + resolve modal
- frontend/src/pages/ObservabilityPage.tsx — Full observability dashboard
- frontend/src/pages/MonitorPage.tsx — Live event stream + run history
- frontend/src/pages/TemplatesPage.tsx — Pre-built template browser

Frontend — Components

- frontend/src/components/AgentBuilder/AgentForm.tsx — 6-section collapsible form
- frontend/src/components/Layout/Sidebar.tsx — Grouped nav with section headers
- frontend/src/components/Monitor/MonitorPanel.tsx — Run history + live event log
- frontend/src/components/WorkflowCanvas/ — Existing canvas wrapper

Infrastructure

- docker-compose.yml — api + frontend + postgres + redis (+ optional MCP services)
- .github/workflows/ — CI: lint → test → docker-build (ARB-007)
- backend/requirements.txt — All Python dependencies incl. httpx, websockets, tiktoken
- frontend/package.json — React + @xyflow/react + lucide-react + recharts

12. Quick-Start Guide

Prerequisites

- Docker 24+ and Docker Compose v2
- OpenAI API key (or compatible endpoint)
- Optional: Telegram bot token for omnichannel integration

Start the Platform

```
git clone https://github.com/org/yuno-ai-platform
cd yuno-ai-platform
cp .env.example .env          # add OPENAI_API_KEY
docker-compose up --build
```

Services come up on:

- Frontend → <http://localhost:3000>
- Backend API → <http://localhost:8000/docs>
- PostgreSQL → localhost:5432
- Redis → localhost:6379

First Steps

1. Open <http://localhost:3000> → Agents → Create Agent. Select memory strategy, enable HITL if needed.
2. Go to MCP Servers → Add a server (stdio or SSE). Click Connect to discover tools.
3. Go to Workflow Builder → Drag nodes onto canvas, connect them, save topology.
4. Go to Workflows → Run a workflow. Watch the Observability Dashboard for live events.
5. If HITL is enabled, go to HITL Approvals when the workflow pauses and review.